GRIFFITH COLLEGE DUBLIN
GRIFFITH COLLEGE CORK
GRIFFITH COLLEGE LIMERICK

QUALITY AND QUALIFICATIONS IRELAND

EXAMINATION

BACHELOR OF SCIENCE IN COMPUTING

STAGE I

COMPUTER PROGRAMMING
Module code: BSCO-CP

BACHELOR OF SCIENCE (HONS) IN COMPUTING SCIENCE

STAGE I

COMPUTER PROGRAMMING
Module code: BSCH-CP

Lecturer(s):

External Examiner(s):

Date: August 2022 **Time:**

THIS PAPER CONSISTS OF SIX QUESTIONS

FIVE QUESTIONS TO BE ATTEMPTED

SECTION A – COMPULSORY

SECTION B – FOUR QUESTIONS TO BE ATTEMPTED

SECTION A – COMPULSORY

QUESTION 1

a) Write Boolean expressions for the following (where A, B, C, D, E, F, G are integers):

i. A is a positive even number that is divisible by 11.

(A>0 && A%2==0 && A%11==0)

ii. B is a number divisible by 11 but not by 752.

(B%11==0 && B%752!=0)

iii. C is either both less than 55 and greater than 33 or otherwise equal to -35.

((C>33 && C<55) || C==-35)

iv. The value of D is a positive two-digit integer.

(D>=10 && D<=99)

v. E is less than or equal to the cube of F but is not the square of G.

(E<=F*F*F && E!=G*G)

(10 marks)

b) Explain the differences between compile-time errors and run-time errors. Give an example of each type of error, explaining your example.

Run time errors occur during the run time of the program. The code compiles correctly but crashes while running. **[1.5 marks]**

Compile time errors are violations of the syntax of the language that are detected by the compiler. These have to be fixed before the code can be compiled. **(1.5 marks)**

An example of a run-time error might be: **(1 mark)**

```
int [] intArray = new int[10];
 for (int i=0; i<=10; i++)
          intArray[i]=i;
```

since the fact that there is no element at index 10 to assign will only be caught at runtime.

An example of a compile-time error might be: **(1 mark)**

```
public static void main( String [ args) {
```

since we are missing a second "]" bracket for the args parameter

(5 marks)

c) Study the following code section carefully and complete the task below.

```
public static int zeroLowerMatrix(int [][] mat) {
    for (int row = 0; row < mat.length; row++) {
        for (int col = 0; col < row; col++) {
            mat[row][col] = 0;
        }
    }
}
```

Your task is to rewrite the code to use while loops and not for loops.

(5 marks)

5 marks:

- For loop initializations moved to before the while loops (1 mark)
- For loop continuation conditions moved into the while loop conditionals (1.5 marks)
- For loop iteration update moved to end of the while loop bodies (1.5 marks)

Sample solution:

```
public static int zeroLowerMatrix(int [][] mat) {
    int row = 0;
    while (row < mat.length) {
        int col = 0;
        while (col < row) {
            mat[row][col] = 0;
            col++;
        }
        row++;
    }
}
```

d) Design a Java temperature conversion method that takes a temperature value (assumed to be in Fahrenheit) and converts it to Celsius. To convert from Fahrenheit to Celsius, one first subtracts 32 from the original temperature and then multiples the resulting value by $\frac{5}{9}$.

(5 marks)

5 marks:

- 3.5 marks for correct conversion

- 1 mark for appropriate method signature
- 0.5 mark if constants used

Sample Solution:

```
public static double convertor (double fahrenheit) {
        double celsius = 0.0;
        final double TEMP_RATIO = 5.0/9.0;
        final double TEMP_OFFSET = 32.0;
        celsius = TEMP_RATIO*(fahrenheit - TEMP_OFFSET);
        return celsius;
```

e) Explain exactly why this code will not work as intended. What two fixes are needed to get the output on the right?

```
public static void main(String[] args)
{
    int count = 0;
    while( count <= 10)
    {
        if(count == 0 || count = 10)
        {
            System.out.println("----------");
        }
        else
        {
            System.out.println(count);
        }
    }
}
```

```
----------
1
2
3
4
5
6
7
8
9
----------
```

(5 marks)

Answer:

Marks (5)

Error one: count is not incrementing **(1)**
Correction: add count++ at the end of the while loop **(1)**
Error two: count = 10 is doing an assignment not a comparison **(2)**
Correction: the comparison needs to be if(count ==0 || count == 10) **(1)**

f) Given the integer array {4,2,15,3,9,3,12,23,43,14,28}, write a loop for each of the following:

1. Print out all 2 digit numbers in the array **(2 marks)**
2. Print out all odd numbers in the array **(2 marks)**
3. Print out the numbers whose word starts with the letter f in the array **(3 marks)**
4. Print out any number divisible by the number preceding it **(3 marks)**

Totasl (10 marks)

Solution:

```java
int[] nums = {4,2,15,3,9,3,12,23,43,14,28};

//two digits
for(int num : nums)
{
    if(num >= 10 && num <= 100)
        System.out.println(num);
}
//odds
for(int num : nums)
{
    if(num%2 != 0)
        System.out.println(num);
}
//starting with f
for(int num : nums)
{
    char first = Integer.toString(num).charAt(0);
    if(first == '4' || first == '5' || num == 14  || num == 15)
    {
        System.out.println(num);
    }
}
//divisible by the number in front
for(int i =1; i < nums.length; i++)
{
    if(nums[i]%nums[i-1] == 0)
        System.out.println(nums[i]);
}
```

Marks (10)

Two-digit loop **(2)**
Odds loop **(2)**
Basic loop for starting with f **(1)**
Loop checks for starting with 4 and 5 **(1)**
Loop checks for 14 and 15 **(1)**
Basic divisible by number in front loop **(1)**
Loop has modulus/division) **(1)**
Starts at 1 or checks greater than zero for first element **(1)**

Total (40 marks)

SECTION B – <u>FOUR QUESTIONS TO BE ATTEMPTED</u>

QUESTION 2

Write a method called *anagramChecker* that takes two Strings as arguments i.e. (takes,set) and returns true if any combination of the letters in the longer String can be used to create the shorter String and false if it can't. The words can be entered in any order so check if the shortest word can be made from the longest. Also ensure you deal with duplicates letters, i.e. today and yay would be false as there's only one y in today.

Example inputs and outputs

```
(today,toy)   true

(there , heather)  true

(pulse,sell)  false
```

(15 marks)

Solution:

```java
public static boolean anagramChecker (String longer,String shorter)
 {
     int found = 0;
     if(shorter.length() > longer.length())
     {
         String temp = longer;
         longer = shorter;
         shorter = temp;
     }
     for(int i= 0; i < shorter.length(); i++)
     {
         String letter = shorter.substring(i,i+1);
         if(longer.contains(letter))
         {
             longer = longer.replaceFirst(letter,"");
             found++;
         }
     }
     if(found == shorter.length())
         return true;
     else
         return false;
 }
```

Marks (15)

Method called anagramChecker **(1)**
Returns a boolean **(1)**
Takes two strings as inputs **(1)**
Uses appropriate string methods to check if longest contains the shortest letters **(5)**
for any attempt, **(1)**
if they just do if(longest.contains(shortest)) **(2)**
if the method is partially right **(3)**
if almost right **(4)**
Checks for duplicate letters **(2)**
Swaps longest and shortest as needed **(2)**
Returns true if shortest can be made from longest **(2)**
Returns false otherwise **(1)**

QUESTION 3

Create an ArrayList named *myList* containing 20 random Integers in the range 1-50. Define a method that takes an Integer ArrayList as an argument, and returns either true or false, depending on the following conditions:

return true; // If the arraylist contains duplicates values

return false; // If the arraylist contains no duplicate values

Invoke this method and verify whether there are any duplicate values in myList, printing the result to console.

(15 marks)

Solution:

```
public static boolean duplicates(ArrayList<Integer> A) {
	boolean duplicates = false;

	for(int i = 0; i < A.size(); i++){
		for(int j = i+1; j < A.size(); j++){
			if(j!=i && A.get(i)==A.get(j)){
				duplicates = true;
			}
		}
	}
	return duplicates;
}


public static void main(String[] args) {
	ArrayList<Integer> myList = new ArrayList<Integer>();
	Random random = new Random();
	for (int i = 0; i<21; i++) {
```

```
            myList.add(random.nextInt(49)+1);
        }

        if (duplicates(myList))
                System.out.println("Dupes found.");
        else
        System.out.println("No dupes found.");

}
```

Marks: 15

Generating random numbers 1-50 **(2)**
Create ArrayList **(2)**
Populate myList with 20 random numbers **(2)**
Create method that takes in ArrayList, returns boolean **(2)**
Iterate through ArrayList checking for duplicates **(2)**
return duplicates boolean **(2)**
Invoke function using myList **(2)**
Print whether duplicates found **(1)**

QUESTION 4

Create a method named *BubbleSort*. The method should take an integer array as an input parameter. Use the Bubble Sort algorithm to sort the given integer array in descending order. The method does not return anything, simply prints both the original array and the sorted array on the screen.

Solution

```
public void BubbleSort (int[] myArray){
        for (int x = 0; x<myArray.length-2;x++){
                for (int y = 0;y<myArray.length-x-2;y++){
                        if (myArray[y]>myArray[j+1]){
                                int temp;
                                temp = myArray[y];
                                myArray[j]= myArray[j+1];
                                myArray[j+1]=temp;
                        }
                }
        }
        for(int x=0;x<myArray.length;x++)
                System.out.print(myArray[x] + “, ”);
        }
}
```

Marks: 15

Correctly instantiate method with int array argument **(1)**
Outer loop **(2)**
Inner loop **(2)**
Conditional **(2)**
Temporary variable **(2)**
Switch current int with next one **(2)**
Switch next int with temp **(2)**
Print unsorted and sorted array **(2)**

(15 marks)

QUESTION 5

Write a program that reads data from a console and writes that data to a sequential text file with class Formatter. The content of the output (format) should be like the data in the table below: The first column should be the account number, second column should be the first name, third column should be last name and last column should be Balance (a double with 2 decimal places precision). When opening your file you need to appropriately deal with SecurityException and FileNotFoundException. When formatting your file, you need to appropriately deal with FormatterClosedException and NoSuchElementException. You do not need to include imports in your answer.

Account	First Name	Last Name	Balance
100	Bob	Blue	24.98
200	Steve	Green	-345.67
300	Pam	White	0.00
400	Sam	Red	-42.16

(15 marks)

Solution:

```
public class CreateTextFile
{
        private static Formatter output;

        public static void main(String[] args)
            {
                openFile();
              addRecords();
```

```
        closeFile();
    }

public static void openFile()
{
        try {
                output=new Formatter("clients.txt");
                }
        catch(SecurityException securityexception)
        {
                System.err.println("Write permission denied. Terminating");
                System.exit(1);
        }
        catch(FileNotFoundException filenotfoundexception)
        {
                System.err.println("Error opening file. Terminating");
                System.exit(1);
        }
}
public static void addRecords()
{
        Scanner input=new Scanner(System.in);
        System.out.printf("%s%n%s%n?", "Enter account number, first name, last name and "
                        + "balance", "Enter end-of-file indicator to end input.");
        while (input.hasNext())
        {
                try
                {
                        output.format("%d %s %s %.2f%n", input.nextInt(), input.next(),
                                        input.next(), input.nextDouble());
                }
                catch (FormatterClosedException formatterclosedexception)
                {
                        System.err.println("Error writing to file. Terminating");
                        break;
                }
                catch (NoSuchElementException elementexception)
                {
                        System.err.println("Invalid input. Please try again");
                        input.nextLine();

                }
                System.out.print("?");
        }
}
public static void closeFile()
{
        if (output != null)
        {
                output.close();
        }
```

```
        }

}
```

Marks: 15

Scanner to get input **(2)**
output=new Formatter("clients.txt") **(2)**
output.format("%d %s %s %.2f%n", input.nextInt(), input.next(), input.next(), input.nextDouble()); **(3)**
Filenotfound Exceptions dealt with properly **(3)**
Formatter closed exceptions dealt with properly **(3)**
Closefile (output.close()) **(2)**

QUESTION 6

Create a class called *Invoice* that a hardware store might use to represent an invoice for an item sold at a store. An Invoice should include four pieces of information as ~~for~~ instance variables—a part number (type String), a part description (type String), a quantity of the item being purchased (type int), and a price per item (double). Your class should have a constructor that initializes the four instance variables. Provide a set and a get method for each instance variable. In addition, provide a method named *getInvoiceAmount* that calculates the invoice amount (i.e., multiplies the quantity by the price per item), then returns the amount as a double value. If the quantity is not positive, it should be set to 0. If the price per item is not positive, it should be set to 0.0. Write a test app named *InvoiceTest* that demonstrates class *Invoice*'s capabilities.

(15 marks)

Solution:

```
public class Invoice {

        private String number;

        private String description;

        private int quantity;

        private double pricePerItem;


        public Invoice(String number, String description, int quantity, double pricePerItem) {

                this.number = number;

                this.description = description;
```

```
        if (quantity > 0) { (2 mark)
                this.quantity = quantity;
        }

        if (pricePerItem > 0.0) { (2 mark)
                this.pricePerItem = pricePerItem;
        }
}

public double getInvoiceAmount() { (1 mark)
        return quantity * pricePerItem;
}

public String getNumber() { (1 mark)
        return number;
}

public void setNumber(String number) { (1 mark)
        this.number = number;
}

public String getDescription() { (1 mark)
        return description;
}

public void setDescription(String description) { (1 mark)
        this.description = description;
}

public int getQuantityPurchased() {(1 mark)
```

```
        return quantity;
    }

    public void setQuantity(int quantity) {(2 mark)
        if (quantity > 0) {
            this.quantity = quantity;
        }
    }

    public double getPricePerItem() { (1 mark)
        return pricePerItem;
    }

    public void setPricePerItem(double pricePerItem) { (2 marks)
        if (pricePerItem > 0.0) {
            this.pricePerItem = pricePerItem;
        }
    }
}
```

Marks: 15 (marks breakdown in the solution program)

(2 mark) if (quantity > 0)

(2 mark) if (pricePerItem > 0.0)

(1 mark) public double getInvoiceAmount()

(1 mark) public String getNumber()

(1 mark) public void setNumber(String number)

(1 mark) public String getDescription()

(1 mark) public void setDescription(String description)

(1 mark) public int getQuantityPurchased()

(2 mark) public void setQuantity(int quantity)

(1 mark) public double getPricePerItem()

(2 marks) public void setPricePerItem(double pricePerItem)